

- * ?Project Hermes Proposed Solution (SIH National Level).
- * The Central Question Hermes Answers:.
- * Not just "Is the engine abnormal right now?" but:.
- * What is its current degradation state, what will happen during the planned mission, why is that outcome predicted, and which feasible operating alternative reduces that risk?.
- * BLOCK 1 Why Static Monitoring Fails (The Problem Frame).
- * Context-Blind Threshold Monitoring is Operationally Dangerous.
- * Legacy aero-engine monitoring uses fixed alarm thresholds.
- * The same EGT value at takeoff (high load, sea level) and at 6,000m cruise means entirely different things.
- * Hermes evaluates every engine reading against condition-aware expected behaviour generated by a physics model not a fixed number.
- * Fixed thresholds that ignore altitude, load, RPM and mission phase will always produce false alarms or missed faults.
- * Hermes does not.
- * BLOCK 2 The Technical Core: Physics + State Estimation Pipeline.
- * 1 PINN + ODE Physics Model.
- * - A Reduced-Order Mean Value Engine Model (MVEM) encodes thermodynamic relationships: combustion efficiency, volumetric efficiency, thermal transfer, lubrication state.
- * - At every timestep, the model outputs expected engine behaviour given current operating conditions (altitude, throttle, RPM, ambient temp).
- * - The physics model is the ground-truth reference the AI continuously checks reality against.
- * - Key claim: Every AI prediction in Hermes is anchored to thermodynamic relationships.
- * The system cannot recommend an action that violates physics.
- * 2 UKF (Unscented Kalman Filter) State Estimator.
- * - Engine dynamics are nonlinear.
- * UKF handles nonlinearity without full model linearization (unlike EKF).
- * - Purpose: Fuse noisy, incomplete sensor telemetry with PINN/ODE predictions to produce smoothed, continuously updated internal engine state estimates.
- * - Estimates latent health variables that cannot be directly measured: combustion efficiency, injector condition, lubrication state, mechanical friction.
- * - Role in pipeline: PINN/ODE ? UKF State Estimation ? Fault Detection ? RUL Prediction.
- * - Why it matters: Downstream ML models receive clean, physically-grounded inputs rather than raw noisy sensor readings.
- * Garbage in, garbage out UKF prevents garbage in.
- * 3 Physics-Based Residual Analysis.
- * - Core diagnostic signal: Residual = Actual Sensor Reading ? Physics-Expected Reading.
- * - A healthy engine produces near-zero residuals across all sensors.
- * - Persistent, multi-sensor divergence triggers the diagnostic chain not a single spike, which could be noise.
- * 4 Individual Engine Fingerprinting.
- * - A generic fleet-average baseline is insufficient.
- * Manufacturing tolerances mean Engine A normal vibration is 1.
- * 5g; Engine B is 1.
- * - Hermes builds a per-engine behavioural baseline using one-class modelling on historical sortie data.
- * - Anomaly detection fires when an engine deviates from its own norm, not a population average.
- * - Outcome: Fewer false alarms, earlier detection of subtle progressive degradation, better RUL accuracy.
- * BLOCK 3 Fault Intelligence: Detection, Diagnosis, Isolation.
- * 1 Three-Way Fault Triage.
- * Residual divergence does not automatically mean engine failure.
- * Hermes classifies into three cases:.
- * Category Signature Pattern Response.
- * Sensor Fault / Drift Single sensor residual spikes; all others nominal Sensor isolation ? Flag as instrumentation fault, not engine fault.
- * Twin Model Mismatch Broad residual increase after operating regime shift Model recalibration trigger

"The twin, not the engine, is wrong".

- * A false alarm grounding a UAV mid-surveillance mission has operational cost equivalent to a real fault.
- * Sensor-engine fault separation is not optional.
- * 2 Zero-Day / Open-Set Fault Detection.
 - * - Supervised XGBoost classifies known fault types (injector degradation, oil starvation, overheating, misfire).
 - * - Isolation Forest runs in parallel as the open-set anomaly detector.
 - * - If telemetry does not match any known fault signature: output is "NOVEL ANOMALY outside training distribution", not a forced misclassification.
 - * - In a safety-critical system, confidently reporting an unknown unknown is more operationally valuable than incorrectly labelling it as a known fault.
- * 3 Multimodal Acoustic + Vibration Fusion (Digital Stethoscope).
 - * - Piezoelectric accelerometers and onboard microphones stream vibration and acoustic data alongside standard telemetry.
 - * - Audio Spectrogram model detects: bearing wear, valve timing drift, combustion knock, mechanical imbalance.
 - * - These signatures appear hours before they cause measurable changes in EGT, CHT or oil pressure.
 - * - The USP: Sub-component fault detection before thermodynamic degradation begins.
- * Hermes does not just read the engine vitals it listens to its heartbeat.
- * BLOCK 4 Health Output.
 - * 1 Engine Health Index (EHI).
 - * - A single composite scalar: 100 (pristine) ? 0 (imminent failure).
 - * - Inputs: UKF-cleaned sensor states, residual magnitudes, degradation trajectory, RUL estimate.
 - * - An operator cannot act on 14 sensor graphs during a high-stress sortie.
 - * One number they can act on immediately.
 - * This is what makes Hermes a health monitoring system, not just a data dashboard.
 - * 2 Uncertainty-Aware RUL via Monte Carlo Dropout LSTM.
 - * - Standard LSTM outputs a single RUL point estimate.
 - * That is not actionable for a safety-critical Return-To-Base decision.
 - * - Hermes uses MC Dropout LSTM (Gal & Ghahramani, ICML 2016): Dropout is kept active at inference.
 - * N=50 stochastic forward passes generate a full predictive distribution.
 - * - Captures both aleatoric uncertainty (model own sigma via NLL training) and epistemic uncertainty (variance across MC passes).
 - * - A pilot can make a life-or-death RTB decision on a confidence interval.
 - * They cannot on a black-box point estimate.
- * BLOCK 5 Mission Intelligence (The Core Differentiator).
 - * The problem statement title includes "Mission Reliability Enhancement.
 - * " Most teams ignore this entirely.
 - * Hermes makes it the centrepiece.
 - * 1 Future Mission Simulation.
 - * - Given: current EHI + planned mission profile (altitude curve, throttle, duration, ambient conditions).
 - * - Hermes simulates the engine degradation trajectory across the entire upcoming mission.
 - * - Output: predicted end-of-mission engine health, RUL margin at completion, Mission Completion Risk Score (0 100%).
 - * - "Before the UAV takes off, Hermes already knows if it can land safely.
 - * 2 Counterfactual "What-If" Prescriptive Engine.
 - * - Does not just report risk.
 - * Asks: what operating changes would reduce it?.
 - * - Simulates alternative mission profiles (varied altitude, throttle, duration) and computes risk delta for each.
 - * - Prescriptive, not just predictive.
 - * Decision support, not autonomous control operator accepts or rejects.
 - * 3 Constraint-Aware Mission Optimization.
 - * - Lowest engine-risk scenario does not equal the correct operational choice.
 - * - System filters counterfactual alternatives against hard mission constraints: minimum surveillance duration,

required altitude corridor, fuel endurance, operating region.

- * - Output: Safest feasible option, not just safest option.

- * BLOCK 6 Defense-Specific Layer.

- * 1 Adversarial Sensor Spoofing Detection.

- * - In contested environments (EW, jamming), sensor signals can be corrupted or spoofed.

- * - Because Hermes maintains a continuously updated physics-based expected state, readings that are thermodynamically impossible trigger a dedicated flag: "Spoofed / corrupted sensor physical impossibility detected" rather than generating a false fault diagnosis.

- * - Example: Oil pressure dropping 40 PSI in 0.

- * 3 seconds violates the thermodynamic model.

- * Hermes flags it as a sensor integrity issue, not an oil pump failure.

- * - This requires a physics model to have a reference for "physically impossible.

- * " Purely data-driven systems cannot detect this.

- * No other team will have thought of it.

- * BLOCK 7 Supporting Architecture.

- * Synthetic Fault Generator (Data Strategy).

- * - Real MALE-UAV piston engine fault data is rare, proprietary, and dangerous to reproduce.

- * - Hermes generates synthetic fault data by injecting faults at the physical parameter level (injector flow coefficient, thermal transfer rate, friction coefficient).

- * - Faulty sensor signatures emerge from the physics they are not artificially added noise.

- * - Outcome: Labeled training data reflecting real causal fault physics, covering fault types that may never have occurred in real fleet history.

- * Role-Based Dashboards.

- * Three separate views not one monolithic screen:.

- * Role Primary Question Key Displays.

- * Propulsion Engineer What is happening and why? Residuals, Actual vs Expected, Drift Score, SHAP Contributions, Degradation Trajectory.

- * Post-Flight Replay & Analysis.

- * - Every sortie is reconstructed through the Digital Twin.

- * - Fault timeline, degradation progression, RUL evolution, and alert history are replayed for post-mission engineering analysis.

- * Full Pipeline (End-to-End).

- * Citation Anchors.

- * Concept Reference.

- * UKF for nonlinear systems Julier & Uhlmann (1997).

- * MC Dropout as Bayesian Approximation Gal & Ghahramani (2016), ICML.

- * Digital Twins in PHM Grieves & Vickers (2017).

- * Hermes: MALE-UAV Aero-Piston Engine AI Digital Twin.

- * Hermes is an AI-enabled, mission-aware Digital Twin designed for aero-piston engines (specifically modeling a naturally aspirated Rotax 914) used in Medium Altitude Long Endurance (MALE) UAVs.

- * Instead of relying on simple rule-based thresholds, Hermes utilizes a hybrid physics and data-driven approach.

- * It generates a continuous expected baseline using a Mean-Value Engine Model (MVEM) and uses Machine Learning to analyze the residuals (Actual - Expected) to detect anomalies, classify faults, and predict Remaining Useful Life (RUL).

- * Key Features.

- * - Real-Time Digital Twin: Continuous state synchronization between physical telemetry and the thermodynamic model.

- * - Residual Analysis & Drift Detection: Employs XGBoost and Probabilistic LSTMs to evaluate divergence from expected physical bounds.

- * - Operator Dashboard (GCS): A Next.

- * js/FastAPI interface for Mission Commanders featuring actionable AI prescriptions and a counterfactual "What-If" mission sandbox for risk evaluation.

- * Documentation & Architecture.
- * This repository is strictly governed by its documentation.
- * All autonomous agents and human contributors must read and adhere to the following before pushing code:.
- * Domain Glossary & Boundaries: Defines the Ubiquitous Language (e.
- * , EHI, Residuals) and strict module boundaries (Simulation, Twin Core, Frontend, ML).
- * Tech Stack & Rules: The explicitly approved stack (FastAPI, Next.
- * js, PyTorch) and anti-patterns (no fixed-threshold alarms).
- * Agent Operating Rules: The strict workflow and version control rules governing the project.
- * Project Roadmap (Wayfinder).
- * The project is actively managed via a local markdown tracker using the Wayfinder protocol, split across 6 distinct domain roles.
- * You can view the current frontier, blockers, and decisions in the Wayfinder Map and the Issue Tracker.
- * Wayfinder Map.
- * Destination.
- * Deliver a fully functional MVP of the MALE-UAV Digital Twin, complete with the synthetic telemetry generator, baseline ML pipeline, and the operator dashboard, divided across 6 domain-specific roles.
- * - Domain: MALE-UAV aero-piston engines (focusing on a naturally aspirated Rotax 914 baseline).
- * - The effort relies on a hybrid data/physics approach: building physical simulation, extracting residuals, and applying ML for fault and Remaining Useful Life (RUL) prediction.
- * Decisions so far.
- * - 01-researcher.
- * md Stage advanced PINN-GAT-ODE after baseline XGBoost/LSTM; integrate as secondary AI module consuming physical residuals, not replacing base physics.
- * - 04-simulation.
- * md Architecture finalized: empirical maps with lag filters, centralized FaultManager, and a Python API for explicit time-stepping and runtime fault injection.
- * 1-simulation.
- * md Phase 2 completed: added ISA atmosphere environment model and dynamic telemetry parameters (engineload, injectiontiming).
- * - 05-datasets.
- * md Telemetry generation pipeline finalized: YAML mission sequences, exponential fault degradation scheduler, automated RUL labeling, and partitioned Parquet export.
- * scratch/datasets/issues/01-baseline-pipeline.
- * md Built generate_baseline.
- * py orchestrator script to export partitioned Parquet healthy telemetry.
- * scratch/datasets/issues/02-fault-scheduler.
- * md Implemented dynamic exponential fault scheduler in generate_mission.
- * scratch/datasets/issues/03-rul-labeling.
- * md Added RUL logic with capping to prevent NaN targets in datasets.
- * scratch/datasets/issues/04-bulk-orchestrator.
- * md Added multiprocessing CLI to rapidly generate perfectly balanced datasets.
- * md Datasets Phase 2: Operational Bounds, Noise, and Environment completed.
- * scratch/datasets/issues/phase 2/05-sensor-noise.
- * md Added sensor noise to simulation and timesincefault_injection metadata.
- * scratch/datasets/issues/phase 2/06-weather-bounds.
- * md Enabled ambienttempoffset and bounds sampling [min, max] for mission configurations.
- * scratch/datasets/issues/phase 2/07-base-missions.
- * md Created takeoff_cruise.
- * yaml & loiter.
- * yaml and refactored bulk generator to use them.
- * - 03-ml-engineer.
- * md ML pipeline Stage 1 completed: scripts written for residual generation, feature engineering, and

training XGBoost/Isolation Forest.

- * 1-ml-engineer-stage-2.

- * md ML Stage 2 completed: implemented probabilistic LSTM for RUL prediction using PyTorch, optimized via Negative Log-Likelihood.

- * 2-ml-training-guide.

- * md ML Training Execution Guide: documented step-by-step commands, time estimates, default hyperparams, and RAM limits for offline model training.

- * 2-simulation-cylinder-faults.

- * md Simulation Phase 3 completed: added individual cylinder EGTs and windmilling.

- * 3-datasets-cascading-faults.

- * md Datasets Phase 3 completed: cascading secondary faults and updated ML schema generation.

- * 3-simulation-remediation.

- * md Simulation Remediation completed: added EngineFailureException with physical failure thresholds (RPM < 1000, CHT 250°C, Oil Pressure < 20 psi, EGT 900°C, Vibration 0.

- * 9) that terminate simulations early, and run_pipeline catches the exception to anchor RUL to the true moment of engine death.

- * 4-datasets-remediation.

- * md Datasets Remediation completed: added dynamicmaneuvers.

- * yaml with 9 short aggressive alternating phases for transient training data, added "compound" to allclasses in the bulk orchestrator, and updated FaultScheduler to guarantee 2 overlapping fault injections (100% probability) when forcefaultclass="compound".

- * 3-ml-remediation.

- * md ML Remediation completed: XGBoost migrated to MultiOutputClassifier, Isolation Forest tightened to healthy-only training, LSTM retrained on physically-terminating datasets with TelemetryDataset using pre-computed RemainingUsefulLife column.

- * - 07-ml-data-final-fixes.

- * md Final Logic Fixes completed: fixed right-censored RUL contamination for LSTM, prevented primary fault masking in compound scenarios for XGBoost, and aligned Djibouti scenario RUL anchor with exact engine seizure time.

- * - 02-frontend.

- * md Frontend: Operator Dashboard MVP completed: Vite-based React application created and fully configured by user.

- * 4-simulation-interactive-attack.

- * md Simulation Phase 4 completed: interactive attack simulator with CLI menu looping, all 6 fault types, 4 timing modes (interactive/random/fixed/progressive), compound fault stacking with concurrent independent degradation, and FaultManager.

- * update_params() API.

- * 115 unit tests passing.

- * Open Tickets (The Frontier).

- * - 06-integration.

- * md Integration: Real-Time Backend & MQTT (Issues 01 06 COMPLETE).

- * scratch/integration/issues/01-mqtt-broker-sim-playback.

- * md DONE: Embedded MQTT broker (integration/broker.

- * py) and simulation publisher (integration/simpublisher.

- * py) with adjustable speed playback of djiboutialigned.

- * scratch/integration/issues/05-dynamic-ehi.

- * md DONE: Dynamic EHI computed from twindriftscore in useEngineStore.

- * scratch/integration/issues/06-dynamic-physics-baselines.

- * md DONE: ML subscriber broadcasts dynamic expected_* baselines; frontend consumes them, eliminating false CRITICAL alarms.

- * 1-integration-ehi-fix.

- * md Integration: EHI & Dashboard Critical Status Fix (COMPLETE).

- * Not yet specified.

- * - The precise fault severity mappings (e.
- * , how injector flow coefficient maps to physical wear).
- * - The exact layout and state management approach for the What-If sandbox in the frontend.
- * - The deployment architecture for the live backend components (e.
- * , Dockerization strategy).
- * Out of scope.
- * labels: ["in-progress"].
- * [wayfinder:task] Simulation: Engine Physics & Twin Core.
- * Problem Statement.
- * The user needs a physically plausible simulation of a Rotax 914 engine to generate baseline telemetry and synthetic fault data.
- * This data is critical for training and evaluating predictive analytics models (such as diagnostics and Remaining Useful Life (RUL)) for a MALE-UAV Digital Twin, as relying solely on real-world anomalous flight data is impractical, scarce, and dangerous to acquire.
- * A Python-based simulation engine using a Mean-Value Engine Model (MVEM) approach.
- * It will use empirical lookup tables combined with first-order lag filters to model steady-state and transient behaviors without the computational overhead of solving complex ODEs.
- * The simulation will provide a Python API to explicitly step through time, driven by scriptable mission profiles.
- * A centralized FaultManager will allow users to dynamically inject faults at runtime, predictably altering the telemetry to reflect conditions like misfires, sensor drift, and cooling degradation.
- * User Stories.
- * As an ML Engineer, I want the simulation to output baseline telemetry (RPM, CHT, EGT, Oil Press/Temp, Fuel Flow, Vibration, Battery, Injection), so that I can train models on normal operating conditions.
- * As a Data Scientist, I want to load scriptable mission profiles (throttle, altitude, ambient temp), so that I can generate reproducible operational scenarios.
- * As an ML Engineer, I want to explicitly step the simulation time forward via a step(dt) API, so that I can generate training datasets significantly faster than real-time.
- * As an ML Engineer, I want to query the current telemetry state via a get_state() API, so that I can capture the output at any point in the time series.
- * As an Operator, I want the simulation to realistically model transient responses using first-order lag filters, so that the engine doesn't instantly snap to new states when throttle changes.
- * As a Tester, I want to dynamically inject a "sensor drift" fault via the API, so that I can evaluate if the predictive model catches gradual sensor degradation over time.
- * As a Tester, I want to dynamically inject a "cooling degradation" fault, so that I can simulate a blocked radiator and see CHT and Oil Temp rise.
- * As a Tester, I want to dynamically inject a "misfire" fault, so that I can observe the effect on RPM stability, EGT, and vibration.
- * As a Tester, I want to inject "injector abnormalities", so that I can see corresponding changes in fuel flow and EGT.
- * As a Tester, I want to simulate "lubrication issues", so that I can observe drops in oil pressure and spikes in oil temperature.
- * As an ML Engineer, I want vibration to be modeled as an abstract 0.
- * 0 severity index, so that I have a lightweight representation of mechanical health without the overhead of processing high-frequency audio/accelerometer data.
- * Implementation Decisions.
- * - A main Simulation class will serve as the primary API boundary, exposing loadprofile(profiledata), step(dt), getstate(), and injectfault(fault_type, kwargs) methods.
- * - The core physics will rely on empirical maps (e.
- * , dictionaries or data structures mapping throttle and altitude to steady-state target values for temperatures and pressures).
- * - Transient states (like temperatures taking time to heat up/cool down) will be managed by a LagFilter utility class that implements an exponential moving average based on the time step dt and a physical time constant.

- * - A centralized FaultManager will intercept calculations.
- * When step(dt) is called, the Simulation will ask the FaultManager for active fault modifiers and apply them to the current state (e.
- * , adding an offset to the final sensor output, or changing the time constant of a lag filter).
- * - Mission profiles will be defined as structured objects (e.
- * , parsed from JSON or YAML) containing arrays of timestamped setpoints.
- * The simulation will linearly interpolate between these setpoints during step(dt).
- * - Vibration will not use FFTs or time-domain waveforms; it will simply be a derived value based on engine speed and active mechanical faults.
- * Testing Decisions.
- * - What makes a good test: Tests will solely assert against the external behavior (the telemetry dictionary output from getstate()) in response to public API inputs (step, injectfault, load_profile).
- * We will avoid testing the internal state of the FaultManager, the precise values inside the empirical maps, or the internal state variables of the filters.
- * - Tested Modules: The public interface of the Simulation engine package.
- * - Prior Art: (None yet; this is the foundational simulation component).
- * - Test Scenarios:.
- * - Baseline Test: Initialize simulation, step for 100 seconds at a constant 50% throttle, assert CHT and EGT stabilize within plausible bounds and do not oscillate.
- * - Transient Test: Step at 50% throttle until stable, instantly change throttle to 80%, step for 5 seconds, assert CHT is rising but has not instantly snapped to the new maximum (verifying lag filters).
- * - Fault Test: Step to steady state, invoke sim.
- * injectfault('coolingdegradation'), step for another 50 seconds, assert CHT and Oil Temp are strictly greater than the baseline steady state.
- * Out of Scope.
- * - Real-time message brokering (MQTT/ZeroMQ).
- * The simulation only provides a synchronous Python API.
- * A separate integration script/module will be responsible for calling step() and publishing the outputs to a broker.
- * - Full 3D kinematic modeling of the drone airframe (we are exclusively modeling the engine).
- * - High-frequency time-domain vibration audio/accelerometer data.
- * - Solving complex differential equations for physical thermodynamics (we are substituting this with maps and filters).
- * Implementation Status.
- * All core user stories (1 11) are implemented on feature/simulation-foundation with 38 passing tests.
- * See simulation/README.
- * md for full API docs and data generation guide.
- * Issue Status What.
- * 01 Core Physics Done Simulation, LagFilter, empirical maps, step(dt), get_state().
- * 03 Fault Manager Done FaultManager, sensordrift, coolingdegradation.
- * 04 Advanced Faults Done misfire, injectorabnormalities, lubricationissues, vibration_index.
- * Known Gaps (pre-data-generation).
- * - User Story 2 ambient temperature: Profiles currently interpolate throttle and altitude only.
- * Ambient temp is not an input axis in the steady-state maps.
- * - User Story 1 "Injection" channel: Not a separate telemetry output; injector health is reflected through fuelflow and EGT changes via the injectorabnormalities fault.
- * - Sensor noise: No stochastic noise model.
- * The sim is fully deterministic (as required), but realistic training data may need seeded Gaussian noise on sensor outputs.
- * - File-based profile loading: load_profile accepts a Python dict.
- * YAML/JSON file parsing is not built in the caller reads the file.
- * - Parquet export: Not implemented.
- * The README documents CSV export; Parquet is listed in the tech stack as an approved format.

* Further Notes.

* - The simulation must remain fully deterministic.

* Given the same random seed (if any statistical noise is applied) and the exact same mission profile and fault injection sequence, the engine must produce an identical telemetry timeseries on every execution.

* labels: ["done"].

* [wayfinder:task] Simulation: Phase 2 (Advanced Data Generation).

* Following the foundational simulation implementation, gaps were identified between the generated telemetry parameters and the full theoretical set proposed in the docs/archive/master.

* md documentation.

* The ML and Data Science teams require complete operating context environmental variables and derived load metrics to properly contextualize engine behavior.

* However, these additions must not unnecessarily complicate the core simulation loop or compromise the separation of control inputs from measured engine telemetry.

* We will extend the simulation API to dynamically derive the missing environmental and operational parameters using established physical approximations, without altering the underlying steady-state maps or adding new lag filters.

* The International Standard Atmosphere (ISA) model will be used to derive environmental conditions from altitude.

* Engine load and injection timing will be empirically approximated based on current RPM and throttle.

* We intentionally restrict our fault catalog to the 5 baseline faults, declining to add more complex mechanical/combustion faults at this stage to keep the baseline data generation stable.

* As a Data Scientist, I want to query environmental conditions (ambienttemperature, ambientpressure, airdensity) via a separate getenvironment() API, so that I can contextualize engine performance without mixing environmental truths into the engine's internal telemetry output.

* As a Data Scientist, I want environmental variables to be automatically derived from the altitude control input using the standard ISA model, so that I don't have to manually script complex atmospheric profiles.

* As an ML Engineer, I want the getstate() telemetry to include engineload and injection_timing, so that my models have access to these key performance indicators.

* As an ML Engineer, I want engineload and injectiontiming to be instantly calculated from current throttle and rpm, so that they reflect instantaneous operating conditions without the lag associated with thermal measurements.

* - Control Inputs Separation: throttle and altitude will remain strictly as control inputs and will not be output directly in get_state().

* - ISA Environment Model: A new getenvironment() method will be added to the Simulation class.

* It will calculate ambienttemperature (lapse rate from sea level), ambientpressure (barometric formula), and airdensity (ideal gas law) dynamically from the current _altitude.

* - Dynamic Telemetry: engineload and injectiontiming will be calculated inside get_state() using empirical formulas based on throttle and rpm.

* These do not need new lag filters.

* - Fault Scope: No new faults (mechanicaldegradation, combustioninstability) will be added.

* The existing 5 faults (sensor drift, cooling degradation, misfire, injector abnormalities, lubrication issues) are confirmed as sufficient.

* - What makes a good test: Tests should verify the math of the ISA model (e.

*, standard day at sea level yields ~15°C and 101.

3 kPa) and the behavior of the new empirical telemetry calculations.

* - Tested Modules: The Simulation class (getenvironment() and getstate()).

* - ISA Sea Level: Set altitude to 0, assert get_environment() returns standard sea-level values.

* - ISA Altitude: Set altitude to 10,000 ft, assert temperature drops to approximately -5°C and pressure drops accordingly.

* - Engine Load Limits: Step at 0% throttle, assert engine load is near minimum.

* Step at 100% throttle, assert engine load is near 100%.

* - Weather conditions beyond the standard ISA model (e.

*, humidity, non-standard temperature days).

- * - Implementing new fault types beyond the original 5.
- * - The ISA environment values will be derived purely algebraically, without storing hidden states across steps.
- * This maintains the purely deterministic and memoryless nature of the environmental conditions.
- * [wayfinder:task] Simulation: Phase 3 (Cylinder Specific Faults & Windmilling).
- * To properly support generating datasets that accurately reflect historical catastrophic failures (like the Djibouti MQ-1B accident), the core engine model needs higher fidelity at the individual cylinder level and handling for catastrophic engine seizure dynamics.
- * Currently, the engine outputs a single global EGT, and faults are broadly applied to the entire engine block.
- * We will update the core simulation to output individual Exhaust Gas Temperatures (egt1 through egt4) that mirror the baseline EGT but can be independently manipulated by faults.
- * We will also introduce a specific cylinder_failure fault that sharply degrades a targeted cylinder, and support RPM coast-down to a windmilling state during total seizure.
- * As an ML Engineer, I want the getstate() telemetry to include egt1, egt2, egt3, and egt_4, so that my models can detect localized cylinder anomalies rather than just global overheating.
- * As a Tester, I want to inject a cylinder_failure fault that specifies a target cylinder, so that I can recreate specific physical failures where a single cylinder's EGT plummets.
- * As a Tester, I want the cylinder_failure fault to also degrade overall engine performance (RPM drop) proportionally, so that the simulation physics remain somewhat realistic.
- * As a Data Scientist, I want the simulation to support a windmilling state where an incapacitated engine coasts down to a minimum RPM (e.
- * , 1000) rather than coming to a dead stop, to reflect realistic in-flight aerodynamics.
- * - Cylinder Telemetry: The egt1 to egt4 telemetry outputs have already been added to the base engine.
- * They currently track the global filtered egt value plus independent sensor noise.
- * - New Fault Type: Implement cylinderfailure in the FaultManager.
- * It should accept a parameter (e.
- * , cylinder=3) and apply a severe negative outputoffset exclusively to that cylinder's EGT.
- * It should also apply a negative target_offset to rpm based on severity.
- * - Windmilling Simulation: Rather than complex aerodynamic modeling, we will rely on injecting sensor_drift on rpm to force the RPM to a windmilling speed (1000 RPM) when the throttle is at 0 and the engine has seized.
- * - What makes a good test: Tests should ensure that individual cylinders can deviate significantly from the baseline EGT while the others remain unaffected.
- * - Tested Modules: Simulation class, FaultManager.
- * - Healthy Cylinders: Ensure all 4 cylinders track within standard noise bounds of the global EGT.
- * - Cylinder Failure: Inject cylinderfailure on cylinder 3.
- * Assert that egt3 drops dramatically while egt1, egt2, and egt_4 remain steady, and overall RPM drops.
- * - Thermodynamic modeling of heat transfer between adjacent cylinders.
- * - True aerodynamic windmilling calculations based on airspeed (we will use fixed RPM overrides).
- * labels: ["complete", "simulation", "04.
- * 3 Simulation Remediation - Engine Failure Thresholds.
- * Context / Problem Statement.
- * Currently, the engine simulation runs for a fixed duration regardless of the severity of the faults injected.
- * When a catastrophic physical fault is injected (e.
- * , severe cooling degradation leading to extreme CHT, or massive misfire causing RPM drop), the simulation continues logging data until the predetermined maxtime.
- * As a result, the downstream ML pipeline calculates the Remaining Useful Life (RUL) as a simple countdown timer (maxtime - current_time), training the LSTM model to falsely believe that catastrophically damaged engines survive just as long as perfectly healthy ones.
- * The simulation engine must be updated to inherently recognize catastrophic failure states.
- * When critical telemetry thresholds are breached (e.
- * , RPM drops below idle, or temperatures exceed structural limits), the simulation should terminate early.
- * This physical termination naturally shortens the generated data file.

- * Consequently, the existing RUL calculation in the ML pipeline will automatically assign correct, responsive labels (reaching 0 at the moment of catastrophic failure).
- * As a Simulation Engineer, I want the engine simulation to halt execution immediately when RPM drops below 1000, so that engine stall conditions are realistically modeled.
- * As a Simulation Engineer, I want the engine simulation to halt execution immediately when Cylinder Head Temperature (CHT) exceeds 250°C, so that catastrophic thermal failure is modeled.
- * As a Simulation Engineer, I want the engine simulation to halt execution immediately when Oil Pressure drops below 20 psi, so that fatal lubrication failure is modeled.
- * As a Simulation Engineer, I want the engine simulation to halt execution immediately when Exhaust Gas Temperature (EGT) exceeds 900°C, so that critical exhaust valve/turbine failure is modeled.
- * As a Simulation Engineer, I want the engine simulation to halt execution immediately when the Vibration Index exceeds 0.
- * 9, so that catastrophic structural failure is modeled.
- * As a Data Engineer, I want the telemetry pipeline (run_pipeline) to gracefully handle a simulation that terminates early, so that it can export a shorter Parquet file without crashing.
- * As an ML Engineer, I want the generated Parquet files for catastrophically failed missions to be physically shorter than healthy missions, so that the dataset.
- * py RUL calculation organically drops to zero at the point of failure.
- * - Modules to be modified:.
- * - simulation/engine.
- * py (Core Simulation class).
- * - datasets/generate_mission.
- * py (Data export pipeline).
- * - Architectural approach:.
- * - Introduce an explicit state-check mechanism within or immediately following the Simulation.
- * step() method.
- * - The Simulation class will raise a specific EngineFailureException (or return an is_alive=False flag) when the state dictionary breaches any of the critical thresholds.
- * - runpipeline will catch this exception (or check the flag) during its numsteps loop and safely break out of the loop early, writing out the telemetry collected up to that exact timestamp.
- * - Critical Thresholds:.
- * - rpm < 1000.
- * - oil_pressure < 20.
- * 0 (and applicable to individual cylinder egt1 to egt4).
- * - vibration_index 0.
- * - What makes a good test: Tests should verify the external behavior of the Simulation and the generate_mission pipeline without relying on the internal state updates (e.
- * we only care that the pipeline halts and output is shorter, not exactly how tau filters reached that point).
- * - Modules to be tested:.
- * - tests/unit/testcorephysics.
- * py (or a new testenginefailure.
- * - tests/unit/testmissionprofiles.
- * py (or similar dataset pipeline test).
- * - Testing Seams:.
- * Simulation Level: Instantiate a Simulation, manually set a state value beyond a critical threshold (or inject a severity 1.
- * 0 fault and step forward), and assert that stepping further raises the failure exception or indicates a dead engine.
- * Pipeline Level: Run generatesinglemission with a catastrophic fault, and assert that the resulting DataFrame length is strictly less than the length expected from max_time / dt.
- * - Modifying the XGBoost or LSTM training scripts (this is handled in Phase 03.
- * - Adding new types of faults or secondary cascade logic (handled in Phase 05.
- * - Creating the dynamic transient mission profiles (handled in Phase 05.

- * - By keeping the thresholds centralized in Simulation, any downstream consumer (like a real-time dashboard or what-if sandbox) will automatically benefit from realistic engine death mechanics.
- * labels: ["ready-for-agent"].
- * [wayfinder:task] Simulation: Phase 4 (Interactive Degradation Simulator).
- * The existing "real case simulator" (the Djibouti accident scenario) is hardcoded to a specific 9-hour flight and fault sequence.
- * Users cannot dynamically generate their own scenarios.
- * We need a flexible, interactive replacement focused on prediction (prognostics/Remaining Useful Life).
- * It should allow the user to choose the types of engine faults (attacks) to inject and simulate them in a progressive degradation manner (gradually worsening over time).
- * This flexibility is necessary to rapidly generate varied failure datasets for testing predictive models and evaluating our multi-output classification and prognostic capabilities.
- * Additionally, the simulator currently does not output any Remaining Useful Life (RUL) metrics, relying on downstream ML components (like an LSTM model).
- * This is problematic when users want to treat the simulator output as ground truth or visualize RUL directly without running the complex ML pipeline, which can lead to unpredictable or delayed evaluations of the RUL.
- * Replace the static accident scenario with a new interactive attack scenario module.
- * This module will present a command-line menu to the user, allowing them to selectively configure one or more faults, define a progressive degradation timing strategy, and execute a standard baseline mission profile.
- * Telemetry will be exported for downstream machine learning evaluation.
- * To resolve the RUL dependency, the script will directly compute and export a hardcoded RUL column in its CSV output based on the injected attack type.
- * To prevent erratic frontend updates, this RUL metric will decrement in 10-second discrete steps rather than updating every second.
- * As a tester, I want to be presented with an interactive command-line menu upon launching the simulator, so that I can configure the simulation without modifying source code.
- * As a tester, I want to see a comprehensive list of all supported engine faults (e.
- * , misfire, sensor drift, cylinder failure), so that I can choose which specific anomaly to inject.
- * As a data scientist, I want to be able to select multiple faults in a single simulation run, so that I can test compound failure conditions and multi-output ML model accuracy.
- * As a tester, I want the simulator to loop the fault selection menu until I explicitly indicate I am done, so that I can add as many overlapping faults as needed.
- * As an ML engineer, I want the injected faults to exhibit progressive degradation (exponentially worsening over time), so that the simulation accurately represents gradual mechanical wear rather than instantaneous failure.
- * As a tester, I want to be prompted to enter a "Target Time to Failure" duration, so that I can control the rate at which the exponential degradation reaches critical severity.
- * As an ML engineer, I want the simulator to automatically run a standardized 1-hour flight profile (Takeoff, Climb, Cruise, Loiter, Return, Land), so that my models are tested against a consistent operational baseline.
- * As a data scientist, I want the simulator to directly embed a hardcoded Remaining Useful Life (RUL) metric into the telemetry CSV, so that I have a reliable ground truth label without relying on a downstream LSTM model.
- * As a tester, I want the hardcoded RUL to start at a high default value (e.
- * , 5000) during healthy engine operation, so that healthy states are easily distinguishable from degraded states.
- * As a tester, I want the RUL to switch to a specific initial value based on the type of attack injected (e.
- * , misfire=1500, cylinder_failure=600), so that different attack severities reflect accurate time-to-failure estimates.